

Resumen de Arquitectura de Computadores

Clase del 12 de mayo - Inverso multiplicativo modular y contexto RISC/ARM

Idea central de la clase

La clase se divide en dos bloques. Primero se revisa el proyecto de programación en ensamblador 80x86-64: calcular el inverso multiplicativo modular usando el algoritmo extendido de Euclides y respetando el uso de módulos con stack frame. Luego se introduce una parte histórica/conceptual sobre RISC y ARM, mostrando por qué ARM aparece como una respuesta de diseño distinta a la tradición CISC/Intel.

1. Bloque principal: proyecto de inverso multiplicativo modular

1.1 Qué debe hacer el programa

- Diseñar y escribir un programa ensamblador 80x86-64 que calcule el inverso multiplicativo modular de un entero.
- El programa debe solicitar al usuario dos enteros: a y p. En el contexto del proyecto, p se usa como módulo.
- Al finalizar, debe indicar si el inverso existe o no. Si existe, debe mostrar algo como: "El inverso de a módulo p es: <n>".
- El proyecto se relaciona con teoría de números y criptografía de llave pública, especialmente con ideas que aparecen en RSA.

Clave para el trabajo

Un inverso modular no siempre existe. Para a módulo p, existe si y solo si $\gcd(a, p) = 1$. Si p es primo, todo número que no sea múltiplo de p tiene inverso modular.

1.2 Fundamento matemático

Teorema del algoritmo extendido de Euclides: para enteros positivos a y b, existen enteros x y y tales que:

$$a*x + b*y = d$$

donde d es el máximo común divisor de a y b.

Conexión con el inverso modular: si se calcula $\gcd(a, p)$ y el resultado es 1, entonces el coeficiente x cumple:

$$a*x + p*y = 1 \rightarrow a*x \equiv 1 \pmod{p}$$

Por eso, x es el inverso multiplicativo de a módulo p. Si x queda negativo, se normaliza sumando p o usando $x \bmod p$.

Ejemplo visto en clase

5 es el inverso multiplicativo de 3 módulo 7, porque $3*5 = 15 = 2*7 + 1$. Entonces $(3*5) \bmod 7 = 1$, y se puede escribir $5 = 3^{(-1)} \bmod 7$.

1.3 Algoritmo extendido de Euclides

La clase revisa el algoritmo en notación Nassi-Schneiderman. La versión práctica para implementarlo puede pensarse así:

```

mcdExtendido(a, b):
  si b = 0:
    d <- a
    x <- 1
    y <- 0
  si no:
    b0 <- b
    x1 <- 0; x2 <- 1
    y1 <- 1; y2 <- 0

    mientras b > 0:
      q <- cociente(a, b)
      r <- residuo(a, b)
      x <- x2 - q*x1
      y <- y2 - q*y1

      a <- b
      b <- r
      x2 <- x1
      x1 <- x
      y2 <- y1
      y1 <- y

    d <- a
    x <- x2
    y <- y2

  si x < 0:
    x <- x + b0

  retornar d, x, y

```

Interpretación: el algoritmo no solo calcula el MCD; también conserva los coeficientes x y y que permiten escribir ese MCD como combinación lineal de los datos originales.

1.4 Ejemplo de seguimiento en Excel

El profesor usó una hoja de cálculo para seguir los valores de a, b, x1, x2, y1, y2, q, r, x, y, d y b0. El ejemplo visible usa a = 36 y b = 60.

Paso	a, b	Cálculo clave	Resultado parcial
Inicio	a=36, b=60	Se preparan x1=0, x2=1, y1=1, y2=0	b0=60
Iteraciones	Se actualizan a<-b y b<-r	q=a div b, r=a mod b	Se recalculan x e y
Final	a=12, b=0	El ciclo termina porque b=0	d=12
Comprobación	36 y 60	$36*2 + 60*(-1)$	12

Ese ejemplo sirve para entender la mecánica del algoritmo. Para el inverso modular, lo importante es que el MCD final sea 1; si el MCD no es 1, no existe inverso modular.

1.5 Requisitos de implementación importantes

- El programa debe estar dividido en módulos. Cada módulo debe ser un procedimiento o función.
- Cada módulo debe ser invocado con su respectivo stack frame.
- No se pueden pasar parámetros ni regresar valores usando registros generales. Los parámetros y retornos deben manejarse con pila, según lo solicitado.
- Para entrada/salida se permite usar bibliotecas del sistema o bibliotecas de C/C++.
- Debe haber una salida clara tanto para el caso donde el inverso existe como para el caso donde no existe.

Advertencia de implementación

Aunque en clases anteriores se usaron registros como RCX, R8 o R9 para pasar valores en ejercicios cortos, este proyecto pide explícitamente stack frame. Por eso, para el proyecto conviene diseñar cada módulo pensando en parámetros, retorno y variables locales en la pila.

1.6 Módulos recomendables para el proyecto

Módulo	Responsabilidad	Dato de salida esperado
leerEnteros	Solicitar/cargar a y p.	a, p
mcdExtendido	Calcular d, x, y usando Euclides extendido.	d, x, y
normalizarModulo	Ajustar x para que quede en el rango $0..p-1$.	$x \bmod p$
evaluarResultado	Decidir si $d=1$. Si $d \neq 1$, no hay inverso.	bandera/resultado
imprimirResultado	Mostrar el mensaje final según corresponda.	mensaje en consola

1.7 Detalles de ensamblador que conviene tener listos

- Para calcular cociente y residuo, se usará división entera. En x86-64, el dividendo para división suele estar repartido en RDX:RAX, por lo que hay que preparar RAX y RDX antes de dividir.
- Si se trabaja solo con positivos, suele limpiarse RDX antes de div. Si se usan enteros con signo, hay que cuidar la extensión de signo antes de idiv.
- El residuo es esencial para actualizar b en Euclides: $b <- r$.
- El ajuste final de x es importante: si $x < 0$, se puede hacer $x <- x + p$, o más robusto, $x <- x \bmod p$.
- El stack frame debe mantenerse consistente: si se reserva espacio con sub RSP, debe restaurarse antes de ret.
- Hay que evitar mezclar tamaños de registros sin cuidado: usar registros de 64 bits para valores de pila/direcciones y tamaños acordes para enteros.

2. Bloque conceptual: RISC, ARM y cambio de paradigma

Después de revisar el proyecto, la clase pasa a una explicación histórica relacionada con el surgimiento de ARM y el movimiento RISC.

2.1 Contexto británico

- El desarrollo de computadoras también empezó con fuerza en Reino Unido, en paralelo con Estados Unidos, dentro del contexto de Church-Turing.
- La Segunda Guerra Mundial afectó ese avance europeo por la destrucción de infraestructura industrial, la desventaja en patentes y la pérdida de imperios coloniales.
- Alrededor de 1950, la industria de computadoras dejó de avanzar con la misma fuerza en Gran Bretaña.

2.2 Acorn y el nacimiento de ARM

- En 1978 se crea Acorn Computers en el llamado "Silicon Fen".

- Acorn produjo computadoras basadas en el MOS 6502 y tuvo éxito en el mercado educativo.
- En 1983, Steve Furber y Sophie Wilson empezaron a trabajar en el diseño de un procesador propio para la compañía.
- ARM1 fue fabricado en 1985. Tenía menos de 25 000 transistores y funcionaba a 4 MHz.
- Para 1989, ARM3 seguía usándose en computadoras para escuelas, pero la compañía quería expandirse a sistemas empujados.
- Acorn participó en un joint venture con Apple, interesada en el mercado de las PDA.
- ARM adoptó un modelo de negocio clave: en vez de vender procesadores, vendía/licenciaba derechos para fabricarlos. Ese modelo fue adoptado luego por otras compañías RISC.

Relación con Arquitectura de Computadores

ARM se vuelve relevante porque representa una filosofía distinta a la evolución histórica de Intel. En lugar de agregar más y más instrucciones complejas, la idea RISC busca instrucciones más simples, regulares y eficientes para facilitar el diseño del hardware y mejorar la ejecución.

3. Preguntas rápidas de repaso

¿Qué calcula el algoritmo extendido de Euclides?

Calcula el MCD de a y b y además encuentra x, y tales que $ax + by = \gcd(a, b)$.

¿Cuándo existe inverso modular de a módulo p ?

Existe cuando $\gcd(a, p) = 1$. Si p es primo, existe para todo a que no sea múltiplo de p .

¿Cuál es el inverso de 3 módulo 7 en el ejemplo?

5, porque $3 \cdot 5 = 15$ y $15 \bmod 7 = 1$.

¿Qué debe pasar si el MCD no es 1?

El programa debe informar que no existe inverso multiplicativo modular.

¿Qué restricción de implementación parece más importante?

Usar módulos con stack frame; no pasar parámetros ni retornar valores usando registros generales.

¿Qué representa ARM en esta parte del curso?

Un ejemplo histórico de arquitectura RISC y de un modelo de diseño más simple/eficiente frente a la tradición CISC.

4. Chuleta de una página para el trabajo

- Entrada: a y p .
- Calcular d, x, y con Euclides extendido aplicado a a y p .
- Si $d \neq 1$: no existe inverso modular.
- Si $d = 1$: el inverso es x normalizado módulo p .
- Mensaje final: "El número no tiene inverso multiplicativo modular" o "El inverso de a módulo p es: $\langle n \rangle$ ".
- Obligatorio: dividir en módulos, cada uno con stack frame.
- Evitar: retornar resultados en R8/R9/RCX si el enunciado exige pila.